KnowAct: Evaluating Functional Theory of Mind through Active Knowledge-State Diagnosis

Anonymous authors
Paper under double-blind review

Abstract

Large language model agents increasingly collaborate with users in learning, research, and decision-making tasks, where effective assistance depends on understanding what the user knows, misunderstands, or remains uncertain about. Existing Theory-of-Mind (ToM) benchmarks primarily evaluate mental-state inference from fixed narratives or conversations, while recent interactive benchmarks emphasize social goal completion, persuasion, or personalization. These settings do not isolate a complementary capability: whether an agent can actively acquire evidence about a user's structured knowledge state and use its evolving user model to decide what to ask next. We introduce KnowAct, a benchmark framework for evaluating functional ToM in knowledge-grounded interaction. Each bounded episode combines a source-grounded domain knowledge graph, a hidden user knowledge map, simulator-only evidence, and a fixed interaction budget. A tested agent selects diagnostic questions, maintains a full-graph working knowledge map, and submits a final reconstruction of the hidden user state. To make evaluation reproducible and auditable, KnowAct separates visible and hidden contexts, constrains the user simulator through a de-identified answer blueprint, and scores profile reconstruction with deterministic node-level metrics rather than an LLM judge. The initial benchmark domain is statistical learning, grounded in an authoritative textbook. This paper presents the benchmark formulation, construction pipeline, simulator design, agent protocol, and evaluation methodology; empirical results are left for subsequent work.

1 Introduction

Language-model agents are moving from single-turn question answering toward long-horizon collaboration. In such settings, the external task state is only part of the problem. An effective agent must also reason about the user: which concepts are already understood, which distinctions are fragile, which misconceptions are present, and what interaction would be most informative or helpful next. This requirement is closely related to Theory of Mind (ToM), broadly understood as reasoning about the mental states of others.

Most machine-ToM evaluations, however, are endpoint prediction tasks. A model reads a narrative or dialogue and answers a question about a character's belief, intention, emotion, or likely behavior (?????). These benchmarks have substantially improved the rigor and breadth of ToM evaluation, but they usually hold the evidence-gathering process fixed: the model cannot choose what to ask, decide which uncertainty to resolve, or update an explicit user model over multiple turns. Moreover, recent results suggest a gap between explicitly inferring a mental state and applying that inference to downstream behavior or judgment (?).

Interactive social-agent environments and applied-ToM methods begin to address this gap. They evaluate goal-oriented dialogue, strategic behavior, persuasion, and user alignment (?????). Yet these settings typically optimize holistic task outcomes or social goals rather than the accurate recovery of a controlled latent user state. Consequently, a successful

interaction may not reveal whether the agent formed an accurate user model, used a shortcut, or merely produced a generally effective response.

We study a narrower but measurable question:

> Can an agent actively infer a user's hidden knowledge state through limited natural-language interaction, and use that evolving estimate to choose subsequent diagnostic actions?

We call this capability functional ToM for knowledge-grounded interaction. The term functional emphasizes that user modeling is evaluated as part of a closed loop: the agent must infer, update, and act, rather than only verbalize a mental-state description. The term knowledge-grounded restricts the hidden state to a reviewed domain representation with diagnosable concepts, making the target explicit enough for controlled evaluation.

To operationalize this question, we introduce KnowAct. Each episode binds four artifacts: (1) an authored domain knowledge graph, (2) a hidden user-specific knowledge map over that graph, (3) grounded evidence used only by the simulator, and (4) an immutable episode configuration including a turn budget and scoring profile. The tested agent sees the public graph and visible dialogue, but not the hidden map or simulator evidence. At each turn it asks one diagnostic question, observes the simulated user's answer, updates a full-graph working map, and selects the next question. At the end of the episode, it submits one mastery prediction for every graph node. This design converts functional user modeling into a partially observed sequential diagnosis problem with a deterministic endpoint target.

Our contributions are:

- We formulate active knowledge-state diagnosis as a controlled evaluation setting for functional ToM, separating mental-state reconstruction from downstream social or task success.
- We introduce a semi-synthetic benchmark construction pipeline that combines source-grounded knowledge graphs, reviewed hidden user maps, evidence-backed simulation, and strict visibility boundaries.
- We define an agent protocol with an explicit full-graph working map and bounded semantic actions, together with deterministic, evidence-auditable reconstruction metrics that avoid an LLM judge in the primary score.

KnowAct does not claim to establish human-like ToM in language models. It operationalizes one practically important behavior: whether an agent can build and use a structured model of a user's knowledge under controlled interaction.

2 Related Work

2.1 Machine Theory-of-Mind Evaluation

Early and contemporary machine-ToM benchmarks commonly frame mental-state reasoning as question answering over stories or conversations. ToMi revisited false-belief evaluation and showed that dataset regularities can inflate apparent ToM performance (?). BigToM introduced causally structured social scenarios and diverse questions about beliefs, desires, and actions (?). FANToM stresses information-asymmetric multi-party conversations and evaluates consistency across multiple question types (?). Hi-ToM extends evaluation to higher-order belief reasoning (?), while OpenToM broadens narratives, personality information, and psychological mental-state questions (?). ToMBench systematizes a wide range of social-cognitive abilities with controlled multiple-choice evaluation (?).

These benchmarks are essential for evaluating whether models can infer latent mental states from supplied evidence. However, they generally do not evaluate the acquisition policy itself: the model receives a fixed context instead of choosing which evidence to request. SimpleToM further exposes an inference–application gap, showing that models that answer explicit mental-state questions correctly may still fail to predict or judge behavior using the

inferred state (?). KnowAct builds on this observation but shifts the application target from behavior prediction to sequential diagnostic action. The agent's user model matters because it determines which question is asked next and which parts of the hidden knowledge map can be recovered within a turn budget.

2.2 Interactive Social Agents and Applied ToM

SOTOPIA evaluates language agents in open-ended social scenarios involving cooperation, competition, negotiation, and relationship management (?). AgentSense expands scenario diversity and evaluates social goal completion and implicit reasoning in interactive settings (?). Applied-ToM methods explicitly insert mental-state reasoning into an agent loop. ToMAgent generates mental-state hypotheses between dialogue turns and learns representations useful for long-horizon social goals (?). ToMELP evaluates an infer–apply loop in route-controlled persuasion (?). ToM-SWE models user goals, preferences, and constraints to improve software-engineering agents under ambiguous and stateful instructions (?).

This line of work establishes that explicit user modeling can improve interaction. Its evaluation target, however, is usually an external outcome such as social goal completion, persuasive effectiveness, or software task success. Such outcomes intentionally combine many capabilities, including planning, generation quality, domain competence, and strategic communication. KnowAct instead isolates the user-modeling loop: the hidden state has an explicit structure, the agent actively probes it, and the final model can be compared node-by-node against ground truth. The two paradigms are complementary. Applied-agent benchmarks ask whether ToM improves task behavior; KnowAct asks whether the agent can construct the user model that such behavior would depend on.

2.3 Personalization, User Profiling, and Knowledge Tracing

Personalization benchmarks evaluate whether language models use user histories or profiles to produce aligned outputs. LaMP provides diverse personalized classification and generation tasks grounded in user-specific records (?). PERSONAMEM evaluates dynamic profiling over long multi-session histories and whether models apply the inferred profile to later responses (?). These benchmarks focus primarily on preferences, traits, and evolving personal context observed passively from prior interactions. In contrast, KnowAct gives the tested agent control over evidence acquisition and focuses on a domain-specific epistemic state.

Educational modeling offers a second close connection. Bayesian Knowledge Tracing models latent learner mastery from sequences of item responses (?); neural approaches such as Deep Knowledge Tracing and Attentive Knowledge Tracing improve predictive flexibility over long response histories (??). LongTutor evaluates long-term evidence acquisition, knowledge diagnosis, and adaptive teaching from expert-annotated learning logs (?). These methods infer learner state from an existing sequence or curriculum. KnowAct instead treats question selection as part of the tested capability: agents ask open-ended natural-language questions, gather qualitative evidence, and reconstruct a full graph state under a shared episode protocol. Thus, KnowAct is closer to active diagnosis than conventional next-response prediction.

2.4 LLM-Based User Simulation

User simulators make interactive evaluation scalable and reproducible, but they can introduce inconsistency, leakage, and simulator-specific artifacts. DuetSim uses separate generation and verification models to improve task-oriented user simulation (?). SimulatorArena directly studies whether LLM simulators are reliable proxies for humans and shows that profile conditioning can improve alignment, while also underscoring the need for explicit simulator validation (?). More broadly, simulator design research emphasizes that a useful simulator must model task-relevant user behavior rather than merely generate fluent turns (?).

Table 1: Positioning of KnowAct relative to adjacent evaluation paradigms. "Active probes" means the tested model chooses what evidence to request.

Paradigm	Interactive	Explicit latent user state	Active probes	Direct state score
Narrative / conversational ToM	Limited	Partial	No	Usually QA
Social-agent arenas	Yes	Usually no	Yes	No
Personalization from histories	Usually no	Profile / preference	No	Sometimes
Knowledge tracing	Sequential	Mastery	Usually no	Prediction-based
KnowAct	Yes	Knowledge map	Yes	Yes

KnowAct adopts an LLM-based simulator but narrows and structures its authority. The simulator does not freely invent a user. It is conditioned on a reviewed hidden map and grounded evidence, and hidden information is transformed into a de-identified answer blueprint before natural-language generation. Primary scoring compares the tested agent's final structured map against the hidden map and does not rely on a simulator or evaluator model to judge success. Human validation of simulator fidelity remains necessary and is treated as an explicit limitation and planned evaluation component.

3 Problem Formulation

3.1 Domain Knowledge Graph

A benchmark domain is represented by an authored knowledge graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \tag{1}$$

where each node $v_i \in \mathcal{V}$ is a diagnosable knowledge unit and each edge $e_{ij} \in \mathcal{E}$ records a reviewed domain relation. The current relation vocabulary includes part_of, prerequisite_for, supports, and contrasts_with. Each node contains a definition, source locators, a diagnostic goal, mastery-level rubrics, diagnostic signals, and simulator guidance. Edges organize the domain and may guide exploration, but they do not themselves encode user state.

3.2 Hidden User Knowledge Map

For a graph with $N = |\mathcal{V}|$ nodes, a hidden user knowledge map is

$$\mathcal{M}^\star = \{s_i^\star\}_{i=1}^N, \tag{2}$$

where each state $s_i^\star$ includes a discrete mastery level $m_i^\star \in \{0, 1, \ldots, 5\}$, evidence references, misconceptions, and unknowns. The six mastery levels are node-specific and are grounded in authored rubrics rather than interpreted as a universal psychometric scale. A map can additionally be paired with a profile context that affects conversational style, but profile information is not itself part of the primary mastery score.

3.3 Bounded Diagnostic Episode

An evaluation episode is specified by

$$\mathcal{E}p = (\mathcal{G}, \mathcal{M}^\star, \mathcal{Z}, T, \rho, \sigma), \tag{3}$$

where $\mathcal{Z}$ is simulator-only evidence, T is the maximum number of interaction turns, ρ is the interaction rule, and σ is the scoring profile. The initial interaction rule permits one primary diagnostic question per turn. At turn t, the tested agent observes the public graph, the visible dialogue history $\mathcal{H}_{t-1}$, and its own working map $\widehat{\mathcal{M}}_{t-1}$. It selects a diagnostic question

$$q_t \sim \pi_\theta(\mathcal{G}, \mathcal{H}_{t-1}, \widehat{\mathcal{M}}_{t-1}), \tag{4}$$

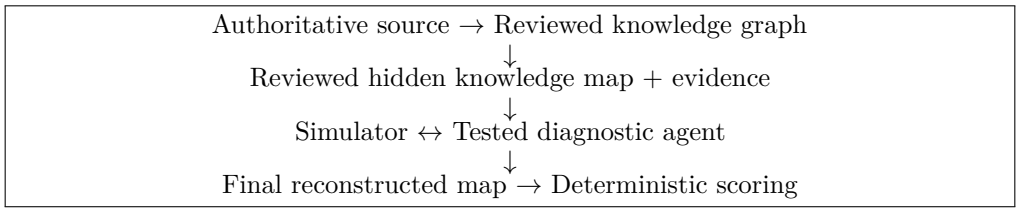

Figure 1: High-level KnowAct pipeline. Benchmark authoring constructs reviewed hidden artifacts; the tested agent interacts only through the visible episode interface.

receives a simulator answer a_t, and updates its estimate

$$\widehat{\mathcal{M}}_t = U_\theta(\widehat{\mathcal{M}}_{t-1}, q_t, a_t). \tag{5}$$

After at most T turns, the agent submits a full-graph reconstruction $\widehat{\mathcal{M}}_T$. The hidden map, evidence, simulator traces, and scoring internals are never visible to the tested agent.

Functional ToM in KnowAct. We define functional ToM operationally as the ability to (i) infer a structured user state from interaction evidence, (ii) maintain and revise that state over time, and (iii) select subsequent actions using the maintained state. The benchmark does not require a particular internal cognitive architecture; it evaluates externally observable behavior under a controlled interface.

4 The KnowAct Benchmark

4.1 Source-Grounded Graph Authoring

The benchmark begins with an authoritative domain source. In the initial version, the domain is statistical learning and the selected source is An Introduction to Statistical Learning with Applications in Python (?). Source material is parsed into ordered segments. A graph-authoring workflow then performs four stages:

1. Node extraction. Segment-level model calls propose diagnosable knowledge units and attach source locators and grounding notes.
2. Reconciliation. Candidate nodes are merged, split, and deduplicated across segments while preserving provenance.
3. Diagnostic enrichment. Each node receives a definition, diagnostic goal, six-level rubric, diagnostic signals, and simulator behavior guidance.
4. Edge proposal and review. Candidate relations are proposed using the complete node set and then manually reviewed before promotion.

The authoring model is therefore a proposal mechanism rather than the source of truth. Candidate artifacts are reviewed and promoted into immutable authored graph files. Requiring source locators reduces unconstrained ontology generation from model memory and supports later audit of node definitions and rubrics.

4.2 Knowledge-Map Authoring

A hidden map assigns one reviewed state to every node in the authored graph. Profile context, background, preferences, and learning goals can be used to generate a coherent candidate user, but the scored target remains the node-level knowledge map. Evidence records make each state behaviorally realizable: they may specify what the user can explain, where an explanation breaks down, which confusion is likely to appear, or what example the user can and cannot produce.

The map-authoring process separates three artifacts:

- the static state used as ground truth for scoring;
- simulator-only evidence that supports natural answers;
- optional profile context that controls wording style but cannot override the state or add unsupported personal facts.

Candidate maps undergo human verification for internal consistency, plausibility, graph coverage, and evaluability before they are eligible for episodes.

4.3 Immutable Episode Manifests

An episode manifest binds the benchmark domain, graph version, hidden map identifier, maximum turns, interaction rule, scoring profile, and pinned model/provider configuration. This prevents accidental changes in graph, map, model, or scoring settings across repeated runs. The runtime maintains separate contexts:

- Tested-agent context: authored graph, interaction rules, visible transcript, and the agent's own working map.
- Simulator context: the current grounded nodes, their hidden states and evidence, visible dialogue needed for continuity, and optional style context.
- Evaluation context: hidden map, final reconstruction, and deterministic scoring configuration.

This separation is central to the benchmark: it makes the tested agent's information boundary explicit and reduces hidden-state leakage through shared prompts or debug traces.

5 User Simulator

The simulator must satisfy two competing requirements: answers should be natural enough to support open-ended diagnosis, yet constrained enough to remain faithful to the hidden map. KnowAct implements the simulator as a staged information boundary rather than a single unconstrained role-playing prompt.

5.1 Question Grounding

Given the current question, visible graph, and limited visible dialogue, a grounder identifies the directly targeted node set $S_t \subseteq \mathcal{V}$. It also detects whether the turn contains multiple independent questions, requests hidden benchmark labels, or cannot be grounded. Crucially, grounding has no access to the hidden map or hidden evidence. This prevents the simulator from interpreting a vague question in whichever way would most conveniently reveal hidden state.

A question may target multiple nodes only when it forms one integrated diagnostic probe, such as asking for a comparison between two methods. Multiple independent questions are rejected or clarified because they would violate the one-question-per-turn budget.

5.2 Hidden Context Selection and Answer Policy

Only after grounding does the context builder retrieve hidden state and evidence, and only for nodes in S_t. Neighboring nodes are not exposed merely because they are connected in the graph. An answer policy then maps the selected hidden information to a structured simulator answer blueprint. The blueprint records a response mode, primary stance, answer shape, and de-identified content units such as:

- a claim the user can express;
- a boundary of partial or fragile understanding;
- a misconception or uncertainty that should surface;

- evidence-supported cues that may be used;
- claims that would overstate the user's ability.

The blueprint intentionally excludes mastery labels, node identifiers, evidence identifiers, map identifiers, user identifiers, and scoring fields. It is the sole content interface to natural-language generation.

5.3 Answer Generation and Leakage Control

The answer generator receives the blueprint, visible dialogue required for continuity, and an optional style hint. It never receives the raw hidden map. Generation is instructed to speak in the first person, avoid benchmark schema language, and preserve uncertainty or misconceptions rather than converting all states into binary know/do-not-know answers. Failed or malformed generations use bounded retries and safe non-leaking fallbacks.

This design does not guarantee that an LLM simulator perfectly matches human behavior. It instead provides a traceable contract: grounding determines which state may be read, policy determines what content may be expressed, and generation controls only surface realization. Hidden policy traces remain available to benchmark authors for audit but are not part of the visible transcript.

6 Tested-Agent Protocol

6.1 Full-Graph Working Map

The tested agent maintains a working assessment for every graph node. Each node record contains:

$$\hat{s}_{i,t} = (\hat{m}_{i,t}, c_{i,t}, n_{i,t}, R_{i,t}), \tag{6}$$

where $\hat{m}_{i,t} \in \{\text{unknown}, 0, \ldots, 5\}$ is assessed mastery, $c_{i,t}$ is diagnostic confidence, $n_{i,t}$ is an optional assessment note, and $R_{i,t}$ is the set of visible turn identifiers supporting the assessment. All nodes begin as unknown. A non-unknown assessment should cite at least one visible observation; unsupported estimates can still be submitted but are reported separately.

A full-graph map serves two purposes. First, it forces the agent to represent uncertainty explicitly instead of silently omitting unvisited concepts. Second, it supports action selection over the entire domain: the agent can compare assessed mastery, confidence, graph position, and evidence coverage when deciding where to probe next.

6.2 Semantic Action Interface

Rather than granting arbitrary state mutation, the runtime exposes a small set of semantic actions:

- read the authored episode graph;
- read the current working map;
- update one node assessment with visible support;
- ask one diagnostic question;
- finalize the reconstructed map.

The interface makes benchmark-relevant behavior observable. A run trace records which nodes were updated, which observations supported those updates, how questions changed over time, and when the agent finalized. It also prevents the evaluated policy from bypassing the intended loop through generic access to hidden artifacts or unrestricted storage.

6.3 Diagnostic Loop

A generic tested agent follows four recurring steps:

1. Interpret evidence: map the latest answer to one or more visible graph concepts.
2. Update beliefs: revise mastery, confidence, notes, and supporting observations.
3. Select a probe: choose a node or relation whose answer is expected to reduce decision-relevant uncertainty.
4. Ask or finalize: generate one grounded diagnostic question or submit the final map when the budget is exhausted.

KnowAct does not prescribe how uncertainty or information value must be computed. A baseline may use a single LLM prompt, while more structured agents may use entropy estimates, graph-aware coverage, Bayesian updates, planning, or learned policies. The shared working-map and action protocol makes these designs comparable.

7 Evaluation

7.1 Primary Reconstruction Metric

The primary metric is episode mastery distance. Let the final predicted mastery for node i be $\hat{m}_i$. For submitted levels 0 through 5, the node loss is squared ordinal distance:

$$d(\hat{m}_i, m_i^\star) = (\hat{m}_i - m_i^\star)^2. \tag{7}$$

An unknownprediction receives the maximum fixed penalty of 36, corresponding to a distance larger than any error between two valid levels. The episode score is

$$D_{\text{mastery}} = \frac{1}{N} \sum_{i=1}^{N} d(\hat{m}_i, m_i^\star), \tag{8}$$

where lower is better. The score is deterministic and requires no evaluator model.

The ordinal loss rewards near misses and penalizes severe over- or under-estimation more strongly than exact-match accuracy alone. It should nevertheless be interpreted relative to each node's authored rubric; the numeric levels are a benchmark encoding, not an assumption that all conceptual differences are psychometrically equal.

7.2 Supporting Metrics

The benchmark can report the following secondary metrics without changing the primary objective:

- exact mastery match rate and per-node signed error;
- missing prediction rate;
- unsupported inference rate, based on absent visible evidence references;
- graph and rubric coverage under a fixed turn budget;
- reconstruction quality as a function of turn count;
- redundant or ungrounded question rate;
- information gain per turn for agents that expose calibrated beliefs.

Misconception and free-text unknown fields are retained for qualitative analysis but are not included in the initial automatic score, avoiding an additional subjective judge.

7.3 Baseline Families

The initial benchmark supports three baseline families:

- Fixed-question agent: follows a predefined diagnostic order and never adapts question choice to answers.

- Random-question agent: samples valid diagnostic targets or questions under episode constraints.
- Simple LLM agent: observes the graph, working map, and visible dialogue, then directly chooses the next question and updates the map through the semantic interface.

These baselines separate the value of interaction from the value of adaptive user modeling. Future methods can add explicit uncertainty estimation, graph-aware planning, information-gain objectives, specialized user-state modules, or multi-agent decomposition while using the same simulator and scoring protocol.

8 Experiments

9 Discussion

9.1 What the Benchmark Measures

A low reconstruction error alone is not sufficient evidence of functional ToM if it is achieved through leakage or a fixed prior. KnowAct therefore couples endpoint accuracy with process observability. The tested agent must operate under a strict visibility boundary, questions consume a finite budget, map updates cite visible turns, and fixed or random policies provide non-adaptive controls. Together, these constraints aim to distinguish three abilities that are often conflated:

1. state inference: interpreting what an answer implies about the user;
2. state maintenance: integrating evidence into a coherent map over time;
3. state-conditioned action: using the current map to select the next probe.

The benchmark is deliberately diagnostic rather than task-complete. It does not yet test whether the recovered map improves teaching, recommendation, or collaborative problem solving. Those downstream tasks can be layered on top once the state-reconstruction loop is validated.

9.2 Relation to Active Learning and Assessment

The interaction can be viewed as active information acquisition over a structured latent state. Unlike pool-based active learning, however, the action is a natural-language question whose interpretation and answer are mediated by a conversational simulator. Unlike computerized adaptive testing, the item set is not necessarily fixed and calibrated in advance; agents may generate integrated conceptual probes. This flexibility motivates the benchmark but also increases variance, making grounding, evidence audit, and simulator validation important.

9.3 Limitations

Simulator fidelity. A simulator conditioned on a reviewed map can be internally consistent without matching how real users express partial knowledge. Human studies are required to measure response similarity, diagnostic usefulness, and whether agent rankings transfer from simulated to human interaction.

Construction circularity. If related model families author the graph, generate the hidden map, simulate the user, and act as the tested agent, correlated biases may make the benchmark easier or distort rankings. Source grounding, human promotion, model-family diversity, rule-based fixtures, and human validation can reduce but not eliminate this risk.

Simplified knowledge state. A static L0–L5 state per node cannot capture all aspects of human knowledge, including context-dependent performance, confidence, procedural fluency, forgetting, and learning during interaction. The initial benchmark freezes the hidden state during an episode to make reconstruction identifiable. Dynamic learning is an important extension.

Single-domain initial scope. The first release focuses on statistical learning. Cross-domain evaluation is needed to determine whether agent policies transfer beyond one graph topology, source style, and concept distribution.

Operational rather than cognitive claims. Success on KnowAct demonstrates behavior compatible with structured user modeling under benchmark constraints. It does not establish that a model possesses human-like beliefs, consciousness, or a psychologically faithful ToM mechanism.

10 Ethical Considerations

User modeling can improve accessibility, tutoring, and collaborative assistance, but it can also enable manipulation, surveillance, or unjustified inference about individuals. KnowAct uses synthetic or explicitly reviewed benchmark profiles and should not be interpreted as permission to infer sensitive attributes from real users without consent. Systems derived from this work should minimize retained personal data, expose uncertainty, allow users to inspect or correct inferred states, and avoid using knowledge-state estimates for high-stakes decisions without appropriate human oversight. Benchmark releases should also document the provenance and licenses of source material and avoid embedding identifiable information in profile contexts or evidence records.

11 Conclusion

We presented KnowAct, a benchmark framework for evaluating functional Theory of Mind through active knowledge-state diagnosis. The framework gives a tested agent a reviewed domain graph but hides a user-specific knowledge map. Under a bounded interaction budget, the agent must ask diagnostic questions, update an evidence-backed working map, and reconstruct the hidden state. A visibility-separated simulator and deterministic scoring protocol make the process auditable and reproducible. By isolating active user-state acquisition from broader social or task success, KnowAct provides a controlled foundation for comparing agent architectures that infer, maintain, and act on models of users. The next step is empirical validation across agent baselines, simulator designs, human participants, and additional knowledge domains.

A Implementation-Oriented Artifact Schemas

The following abbreviated schemas summarize the current benchmark contracts. Exact field names may evolve during implementation, but the separation between public graph, hidden map, visible observations, and final reconstruction should remain stable.

A.1 Knowledge Node

```
{
  "id": "active_learning",
  "name": "Active Learning",
  "type": "concept",
  "definition": "...",
  "source_locators": [...],
  "diagnostic_goal": "...",
  "levels": {"L0": "...", ..., "L5": "..."},
  "diagnostic_signals": [...],
  "simulator_behavior": "..."
}
```

A.2 Hidden User State

```
{
```

```
  "node_id": "active_learning",
  "mastery_level": "L2",
  "evidence_refs": ["ev_104"],
  "misconceptions": [...],
  "unknowns": [...]
}
```


A.3 Agent Working Assessment


```
{
  "node_id": "active_learning",
  "assessed_mastery_level": "L2",
  "diagnostic_confidence": "medium",
  "assessment_note": "Understands the motivation but not
                      query-strategy trade-offs.",
  "supporting_turn_ids": ["turn_03"]
}
```


B Planned Empirical Questions

The empirical study should be designed around questions that directly test the benchmark claim rather than only comparing foundation-model capability:

1. Do adaptive agents reconstruct hidden maps more accurately than fixed and random diagnostic policies under equal turn budgets?
2. Does an explicit working map improve reconstruction and question efficiency relative to transcript-only prompting?
3. Do graph relations improve target selection, or do they induce unsupported propagation between neighboring nodes?
4. How sensitive are rankings to simulator model, answer-policy implementation, question grounding, and profile style?
5. How well do simulator answers and agent rankings agree with matched human-user episodes?